Status	Finished
Started	Thursday, 29 October 2026, 6:46 PM
Completed	Thursday, 29 October 2026, 7:12 PM
Duration	25 mins 44 secs
Grade	**9.00** out of 10.00 (**90**%)

Question **1**

Correct

Mark 1.00 out of 1.00

A sentence is a list of words that are separated by a single space with no leading or trailing spaces. Each word consists of lowercase and uppercase English letters.

A sentence can be shuffled by appending the 1-indexed word position to each word then rearranging the words in the sentence.

For example, the sentence "This is a sentence" can be shuffled as "sentence4 a3 is2 This1" or "is2 sentence4 This1 a3".

Given a shuffled sentence s containing no more than 9 words, reconstruct and return the original sentence.

Example 1:

Input:

is2 sentence4 This1 a3

Output:

This is a sentence

Explanation: Sort the words in s to their original positions "This1 is2 a3 sentence4", then remove the numbers.

Example 2:

Input:

Myself2 Me1 I4 and3

Output:

Me Myself and I

Explanation: Sort the words in s to their original positions "Me1 Myself2 and3 I4", then remove the numbers.

Constraints:

2 <= s.length <= 200

s consists of lowercase and uppercase English letters, spaces, and digits from 1 to 9.

The number of words in s is between 1 and 9.

The words in s are separated by a single space.

s contains no leading or trailing spaces.

Answer: (penalty regime: 0 %)

```
import sys

def sortSentence(s: str) -> str:
    # Split the shuffled sentence into individual words
    shuffled_words = s.split()

    # Create a list placeholder matching the total number of words
    original_words = [""] * len(shuffled_words)

    for word in shuffled_words:
        # The last character is the 1-indexed position
        position = int(word[-1])

        # The actual word content excludes the last digit character
        actual_word = word[:-1]

        # Place the word in its correct 0-indexed position
        original_words[position - 1] = actual_word

    # Join the sorted words back into a single space-separated sentence
    return " ".join(original_words)

def main():
    # Read the full raw line from standard input
    input_data = sys.stdin.read().strip()
    if not input_data:
        return
```

```
28
29      # Process the text and print the reconstructed original sentence
30      result = sortSentence(input_data)
31      print(result)
32
33  if __name__ == "__main__":
34      main()
35
```

	Input	Expected	Got	
✓	is2 sentence4 This1 a3	This is a sentence	This is a sentence	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **2**

Correct

Mark 1.00 out of 1.00

A company wants to send its quotation secretly to its client. The company decided to encrypt the amount they are sending to their client with some special symbols so that the equation amount will not be revealed to any external person. They used the special symbols !,@,#,$,%,^,&,*,>,< for 0,1,2,3,4,5,6,7,8,9 respectively. Write a python code to help the company to convert the amount to special symbols.

(Value rounded off to 2 decimal points)

Input

n: Float data type which reads amount to send

Output

s: : String data type which displays symbols

Sample Testcase 1

Input

10000

Output

@!!!!.!!

Sample Testcase2

1234.56

Output

@#$%.^&

For example:

Input	Result
1345.23	@$%^.#$
15000.59	@^!!!.^<
156789	@^&*><.!!

Answer: (penalty regime: 0 %)

```
import sys

def convert_amount_to_symbols(amount: float) -> str:
    # Rule mapping dictionary
    mapping = {
        '0': '!', '1': '@', '2': '#', '3': '$', '4': '%',
        '5': '^', '6': '&', '7': '*', '8': '>', '9': '<'
    }

    # Format the amount to exactly 2 decimal places as a string
    formatted_amount = f"{amount:.2f}"

    # Translate each character using the mapping dictionary, keeping '.' as is
    symbol_string = "".join(mapping.get(char, char) for char in formatted_amount)

    return symbol_string

def main():
    # Read the input line from standard input
    input_data = sys.stdin.read().strip()
    if not input_data:
        return

    amount = float(input_data)
    result = convert_amount_to_symbols(amount)
    print(result)

if __name__ == "__main__":
    main()

```

	Input	Expected	Got	
✓	1345.23	@$%^.#$	@$%^.#$	✓
✓	15000.59	@^!!!.^<	@^!!!.^<	✓
✓	1234	@#$%.!!	@#$%.!!	✓
✓	156789	@^&*><.!!	@^&*><.!!	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **3**

Correct

Mark 1.00 out of 1.00

In the game of Scrabble™, each letter has points associated with it. The total score of a word is the sum of the scores of its letters. More common letters are worth fewer points while less common letters are worth more points. The points associated with each letter are shown below:

Points Letters

1 A, E, I, L, N, O, R, S, T and U

2 D and G

3 B, C, M and P

4 F, H, V, W and Y

5 K

8 J and X

10 Q and Z

Write a program that computes and displays the Scrabble™ score for a word. Create a dictionary that maps from letters to point values. Then use the dictionary to compute the score.

A Scrabble™ board includes some squares that multiply the value of a letter or the value of an entire word. We will ignore these squares in this exercise.

Sample Input

REC

Sample Output

REC is worth 5 points.

For example:

Input	Result
REC	REC is worth 5 points.

Answer: (penalty regime: 0 %)

```
import sys

def compute_scrabble_score(word: str) -> int:
    # Dictionary mapping each individual letter to its point value
    score_map = {
        'A': 1, 'E': 1, 'I': 1, 'L': 1, 'N': 1, 'O': 1, 'R': 1, 'S': 1, 'T': 1, 'U': 1,
        'D': 2, 'G': 2,
        'B': 3, 'C': 3, 'M': 3, 'P': 3,
        'F': 4, 'H': 4, 'V': 4, 'W': 4, 'Y': 4,
        'K': 5,
        'J': 8, 'X': 8,
        'Q': 10, 'Z': 10
    }

    # Calculate the sum of points for each letter in the upper-cased word
    total_score = sum(score_map.get(letter, 0) for letter in word.upper())
    return total_score

def main():
    # Read the word from standard input
    input_data = sys.stdin.read().strip()
    if not input_data:
        return

    score = compute_scrabble_score(input_data)

    # Display the result in the exact format requested
    print(f"{input_data} is worth {score} points.")

if __name__ == "__main__":
    main()
```

```
32
```

	Input	Expected	Got	
✓	GOD	GOD is worth 5 points.	GOD is worth 5 points.	✓
✓	REC	REC is worth 5 points.	REC is worth 5 points.	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **4**

Correct

Mark 1.00 out of 1.00

A sentence is a string of single-space separated words where each word consists only of lowercase letters.A word is uncommon if it appears exactly once in one of the sentences, and does not appear in the other sentence.

Given two sentences s1 and s2, return a list of all the uncommon words. You may return the answer in any order.

Example 1:

Input: s1 = "this apple is sweet", s2 = "this apple is sour"

Output: ["sweet","sour"]

Example 2:

Input: s1 = "apple apple", s2 = "banana"

Output: ["banana"]

Constraints:

1 <= s1.length, s2.length <= 200

s1 and s2 consist of lowercase English letters and spaces.

s1 and s2 do not have leading or trailing spaces.

All the words in s1 and s2 are separated by a single space.

Note:

Use dictionary to solve the problem

For example:

Input	Result
this apple is sweet this apple is sour	sweet sour

Answer: (penalty regime: 0 %)

```
import sys
from collections import Counter

def uncommonFromSentences(s1: str, s2: str) -> list[str]:
    # Combine words from both sentences into a single list
    all_words = s1.split() + s2.split()

    # Use a dictionary (Counter) to keep track of the frequency of each word
    word_counts = Counter(all_words)

    # Filter and extract words that have a total frequency count of exactly 1
    uncommon_words = [word for word, count in word_counts.items() if count == 1]

    return uncommon_words

def main():
    # Read input lines from standard input
    input_lines = sys.stdin.read().splitlines()
    if len(input_lines) < 2:
        return

    s1 = input_lines[0]
    s2 = input_lines[1]

    result = uncommonFromSentences(s1, s2)

    # Print the space-separated string array output
    print(*(result))

if __name__ == "__main__":
    main()
```

	Input	Expected	Got	
✓	this apple is sweet this apple is sour	sweet sour	sweet sour	✓
✓	apple apple banana	banana	banana	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **5**

Correct

Mark 1.00 out of 1.00

You are given a string `word`. A letter is called **special** if it appears both in lowercase and uppercase in `word`.

Your task is to return the number of **special** letters in `word`.

Constraints

- The input string `word` will contain only alphabetic characters (both lowercase and uppercase).
- The solution must utilize a dictionary to determine the number of special letters.
- The function should handle various edge cases, such as strings without any special letters, strings with only lowercase or uppercase letters, and mixed strings.

Examples

Example 1:

Input: word = "aaAbcBC"
Output: 3

Explanation:
The special characters in `word` are 'a', 'b', and 'c'.

Example 2:

Input: word = "abc"
Output: 0

Explanation:
No character in `word` appears in uppercase.

For example:

Test	Result
`print(count_special_letters("AaBbCcDdEe"))`	`5`

Answer: (penalty regime: 0 %)

Reset answer

```
def count_special_letters(word):
    letters = {}

    for ch in word:
        lower = ch.lower()
        if lower not in letters:
            letters[lower] = [False, False]

        if ch.islower():
            letters[lower][0] = True
        else:
            letters[lower][1] = True

    count = 0
    for lower, cases in letters.items():
        if cases[0] and cases[1]:
            count += 1

    return count
```

	Test	Expected	Got	
✓	`print(count_special_letters("AaBbCcDdEe"))`	`5`	`5`	✓
✓	`print(count_special_letters("ABCDE"))`	`0`	`0`	✓

	Test	Expected	Got	
✓	`print(count_special_letters("abcde"))`	`0`	`0`	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **6**

Correct

Mark 1.00 out of 1.00

Given an array of names of candidates in an election. A candidate name in the array represents a vote cast to the candidate. Print the name of candidates received Max vote. If there is tie, print a lexicographically smaller name.

Examples:

Input : votes[] = {"john", "johnny", "jackie",

"johnny", "john", "jackie",

"jamie", "jamie", "john",

"johnny", "jamie", "johnny",

"john"};

Output : John

We have four Candidates with name as 'John', 'Johnny', 'jamie', 'jackie'. The candidates John and Johny get maximum votes. Since John is alphabetically smaller, we print it. Use dictionary to solve the above problem

Sample Input:

10

John

John

Johny

Jamie

Jamie

Johny

Jack

Johny

Johny

Jackie

Sample Output:

Johny

Answer: (penalty regime: 0 %)

```
import sys
from collections import Counter

def find_winner(votes: list[str]) -> str:
    # Use a dictionary to count frequencies of each vote
    vote_counts = Counter(votes)

    winner = ""
    max_votes = -1
```

```
    for candidate, count in vote_counts.items():
        # Condition 1: Candidate has strictly more votes than current max
        # Condition 2: Candidate has equal votes but is lexicographically smaller
        if count > max_votes:
            max_votes = count
            winner = candidate
        elif count == max_votes:
            if candidate < winner:
                winner = candidate

    return winner

def main():
    # Read all input tokens from standard input
    input_data = sys.stdin.read().split()
    if not input_data:
        return

    # First token is the number of votes, followed by the candidate names
    num_votes = int(input_data[0])
    votes = input_data[1:num_votes + 1]

    result = find_winner(votes)
    print(result)

if __name__ == "__main__":
    main()
```

	Input	Expected	Got	
✓	10 John John Johny Jamie Jamie Johny Jack Johny Johny Jackie	Johny	Johny	✓
✓	6 Ida Ida Ida Kiruba Kiruba Kiruba	Ida	Ida	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **7**

Correct

Mark 1.00 out of 1.00

Objective:

Develop a Python program that takes an input string from the user and counts the number of occurrences of each vowel (a, e, i, o, u) in the string. The program should be case-insensitive, meaning it should treat uppercase and lowercase vowels as the same.

Description:

Vowels play a significant role in the English language and other alphabet-based languages. Counting vowels in a given string is a fundamental task that can be applied in various text processing applications, including speech recognition, linguistic research, and text analysis. The objective of this problem is to create a Python script that accurately counts and displays the number of times each vowel appears in a user-provided string.

Program Requirements:

Input:

First line reading String as input, The string can contain any characters, including letters, numbers, and special characters.

Output:

Display the number of occurrences of each vowel in the string.

The output should list each vowel followed by its count.

Example:

Consider the following example for better understanding:

- **Input:** "Python Programming"
- **Output**

  a = 1
  e = 0
  i = 1
  o = 2
  u = 0

For example:

Input	Result
Hello World	a = 0 e = 1 i = 0 o = 2 u = 0
Python	a = 0 e = 0 i = 0 o = 1 u = 0

Answer: (penalty regime: 0 %)

```
import sys

def count_vowels(text: str):
    # Initialize the vowel dictionary with zero counts
    vowels_count = {'a': 0, 'e': 0, 'i': 0, 'o': 0, 'u': 0}

    # Convert string to lowercase to ensure case-insensitivity
    text_lower = text.lower()

    # Count the occurrences of each vowel
    for char in text_lower:
        if char in vowels_count:
            vowels_count[char] += 1

    # Print the result in the exact format required
    for vowel in ['a', 'e', 'i', 'o', 'u']:
        print(f"{vowel} = {vowels_count[vowel]}")

def main():
```

```
    # Read the full line from standard input
    input_data = sys.stdin.read().strip()
    if not input_data:
        return

    count_vowels(input_data)

if __name__ == "__main__":
    main()

```

	Input	Expected	Got	
✓	Hello World	a = 0 e = 1 i = 0 o = 2 u = 0	a = 0 e = 1 i = 0 o = 2 u = 0	✓
✓	AEIOU aeio u	a = 2 e = 2 i = 2 o = 2 u = 2	a = 2 e = 2 i = 2 o = 2 u = 2	✓
✓	Python	a = 0 e = 0 i = 0 o = 1 u = 0	a = 0 e = 0 i = 0 o = 1 u = 0	✓
✓	abcdefghijklmnopqrstuvwxyz	a = 1 e = 1 i = 1 o = 1 u = 1	a = 1 e = 1 i = 1 o = 1 u = 1	✓
✓	12345!@#$%AEIOU	a = 1 e = 1 i = 1 o = 1 u = 1	a = 1 e = 1 i = 1 o = 1 u = 1	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **8**

Correct

Mark 1.00 out of 1.00

Give a dictionary with value lists, sort the keys by summation of values in value list.

Input : test_dict = {'Gfg' : [6, 7, 4], 'best' : [7, 6, 5]}

Output : {'Gfg': 17, 'best': 18}

Explanation : Sorted by sum, and replaced.

Input : test_dict = {'Gfg' : [8,8], 'best' : [5,5]}

Output : {'best': 10, 'Gfg': 16}

Explanation : Sorted by sum, and replaced.

Sample Input:

2

Gfg 6 7 4

Best 7 6 5

Sample Output

Gfg 17

Best 18

For example:

Input	Result
2 Gfg 6 7 4 Best 7 6 5	Gfg 17 Best 18

Answer: (penalty regime: 0 %)

```
import sys

def main():
    # Read all tokens from standard input
    input_data = sys.stdin.read().splitlines()
    if not input_data:
        return

    # The first line contains the number of entries
    n = int(input_data[0].strip())

    # Parse each subsequent line into the dictionary format
    parsed_items = []
    for i in range(1, n + 1):
        if i < len(input_data):
            parts = input_data[i].split()
            if not parts:
                continue
            key = parts[0]
            # Convert remaining parts to a list of integers
            values = [int(x) for x in parts[1:]]
            total_sum = sum(values)
            parsed_items.append((key, total_sum))

    # Sort the items based on the summation value (ascending order)
    # Python's built-in sort keeps the relative order for equal sums (stable sorting)
    parsed_items.sort(key=lambda item: item[1])

    # Display the result in the exact space-separated format requested
    for key, total_sum in parsed_items:
        print(f"{key} {total_sum}")

if __name__ == "__main__":
    main()
```

```
35
```

	Input	Expected	Got	
✓	2 Gfg 6 7 4 Best 7 6 5	Gfg 17 Best 18	Gfg 17 Best 18	✓
✓	2 Gfg 6 6 Best 5 5	Best 10 Gfg 12	Best 10 Gfg 12	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **9**

Correct

Mark 1.00 out of 1.00

Given a number, convert it into corresponding alphabet.

Input	Output
1	A
26	Z
27	AA
676	YZ

Input Format

Input is an integer

Output Format

Print the alphabets

Constraints

1 <= num <= 4294967295

Sample Input 1

26

Sample Output 1

Z

For example:

Test	Result
`print(excelNumber(26))`	Z

Answer: (penalty regime: 0 %)

Reset answer

```
def excelNumber(num: int) -> str:
    # Lookup string mapping index to characters
    alphabet = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
    result = ""

    while num > 0:
        remainder = num % 26

        # If remainder is 0, it means it's a perfect multiple of 26 ('Z')
        if remainder == 0:
            result = "Z" + result
            num = (num // 26) - 1
        else:
            result = alphabet[remainder - 1] + result
            num = num // 26

    return result

# Verification
# print(excelNumber(26))  # Output: Z
# print(excelNumber(27))  # Output: AA
# print(excelNumber(676)) # Output: YZ

```

	Test	Expected	Got	
✓	`print(excelNumber(26))`	`Z`	`Z`	✓
✓	`print(excelNumber(27))`	`AA`	`AA`	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **10**

Incorrect

Mark 0.00 out of 1.00

Create a student dictionary for n students with the student name as key and their test mark assignment mark and lab mark as values. Do the following computations and display the result.

1.Identify the student with the highest average score

2.Identify the student who as the highest Assignment marks

3.Identify the student with the Lowest lab marks

4.Identify the student with the lowest average score

Note:

If more than one student has the same score display all the student names

Sample input:

4

James 67 89 56

Lalith 89 45 45

Ram 89 89 89

Sita 70 70 70

Sample Output:

Ram

James Ram

Lalith

Lalith

For example:

Input	Result
4 James 67 89 56 Lalith 89 45 45 Ram 89 89 89 Sita 70 70 70	Ram James Ram Lalith Lalith

Answer: (penalty regime: 0 %)

```
n = int(input())
students = {}

for i in range(n):
    data = input().split()
    students[data[0]] = list(map(int, data[1:]))

averages = {}

for name, marks in students.items():
    averages[name] = sum(marks) / len(marks)

max_avg = max(averages.values())
max_ass = max(students[name][1] for name in students)
min_lab = min(students[name][2] for name in students)
min_avg = min(averages.values())

for name in students:
    if averages[name] == max_avg:
```

```
        print(name, end=" ")
print()

for name in students:
    if students[name][1] == max_ass:
        print(name, end=" ")
print()

for name in students:
    if students[name][2] == min_lab:
        print(name, end=" ")
print()

for name in students:
    if averages[name] == min_avg:
        print(name, end=" ")
print()
```

	Input	Expected	Got	
✓	4 James 67 89 56 Lalith 89 45 45 Ram 89 89 89 Sita 70 70 70	Ram James Ram Lalith Lalith	Ram James Ram Lalith Lalith	✓

Your code failed one or more hidden tests.
Your code must pass all tests to earn any marks. Try again.

Incorrect

Marks for this submission: 0.00/1.00.